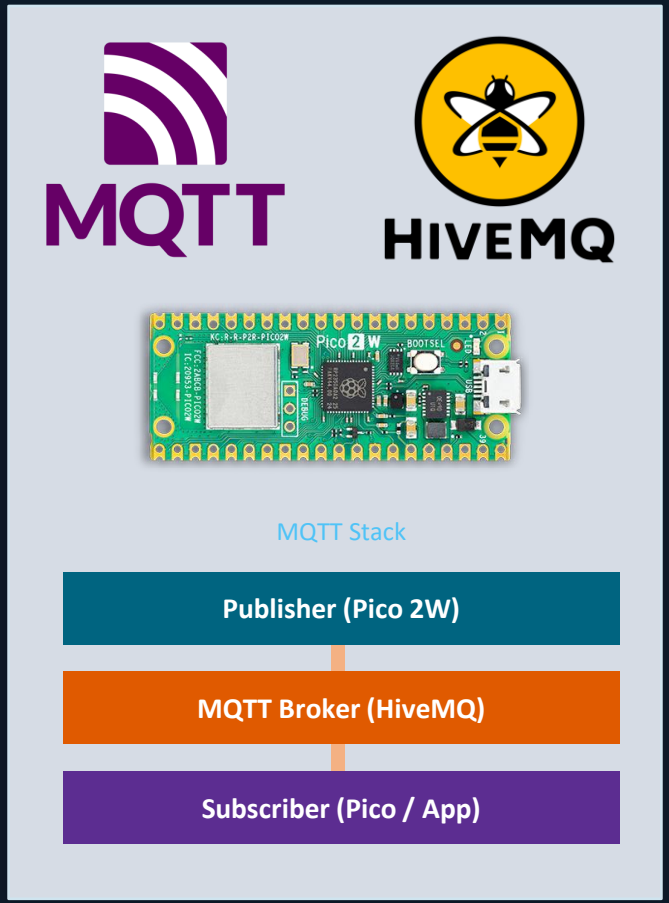
RASPBERRY PI PICO 2W

Getting Started with MQTT

MicroPython · Publish / Subscribe · HiveMQ · BME280 / DHT22 Sensor

Lecture Duration: 3–6 Hours
Level: Intermediate | MicroPython Required

Instructor: Sros Nhek, Ph.D.



Reference: randomnerdtutorials.com/raspberry-pi-pico-w-mqtt-micropython/

Lecture Contents

3–6 Hour Roadmap

01 **Introduction to MQTT**

What it is, why it matters for IoT

02 **MQTT Core Concepts**

Pub/Sub, Topics, Messages, Broker

03 **MQTT Broker Setup**

HiveMQ Cloud, Mosquitto options

04 **Installing umqtt Modules**

simple.py, robust.py via Thonny

05 **Configuration File**

secrets, boot.py, Wi-Fi connection

06 **Publishing Messages**

Sensor data → broker → topic

07 **Subscribing to Topics**

Receive commands, control outputs

08 **Testing & Troubleshooting**

MQTT Explorer, debug tips

Prerequisites & Parts Required

What you need before starting



Raspberry Pi Pico 2W

Wi-Fi-enabled board with MicroPython flashed

Thonny IDE / VS Code

Installed and board recognized over USB

BME280 / DHt22 Sensor

Temperature, Humidity & Pressure (I²C)

Wi-Fi Network

2.4 GHz router with internet access

HiveMQ Account

Free cloud MQTT broker (no install needed)

BME280 Wiring

Pin	Pico 2W
VCC	3.3V
GND	GND
SDA	GP4
SCL	GP5

DHt22 Wiring

Pin	Pico 2W
VCC	3.3V
GND	GND
Data	GP15



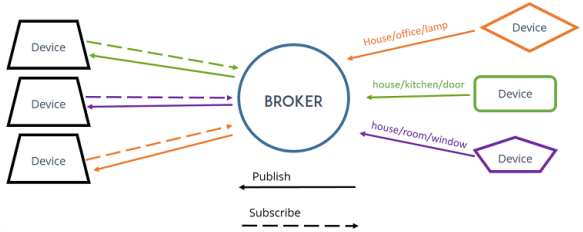
SECTION 01

Introduction to MQTT

What it is · Why IoT needs it · RFC 6455 · History

What is MQTT?

Message Queuing Telemetry Transport



Definition

MQTT stands for **Message Queuing Telemetry Transport**.
It is a lightweight publish-subscribe messaging protocol designed for constrained devices and low-bandwidth, high-latency networks.

Standard:	OASIS Standard & ISO/IEC 20922
Port:	1883 (plain) / 8883 (TLS)
Transport:	TCP/IP
Versions:	3.1.1 (most common) · 5.0
Use Cases:	IoT, Home Automation, Telemetry

Why MQTT for IoT?

Lightweight:
Minimal header overhead (2 bytes min). Perfect for microcontrollers with limited RAM.

Low Bandwidth:
Designed for slow, unreliable networks. Works over 2G/3G, LoRa, Wi-Fi.

Async Delivery:
Broker stores messages for offline devices (with retained messages).

Many-to-Many:
One publisher → broker → many subscribers simultaneously.

Secure:
TLS/SSL encryption + username/password authentication.

HTTP vs MQTT Comparison

Choosing the right protocol for your project

Feature	HTTP	MQTT
Communication	Request–Response (pull)	**Publish–Subscribe (push/pull)**
Connection	New connection per request	**Persistent long-lived connection**
Overhead	~800 B HTTP headers/request	**2–4 byte MQTT frame header**
Server push	Not possible (polling needed)	**Native server-to-client push**
Multi-client	Each client polls separately	**Broker fans out to all subscribers**
Best for	REST APIs, file downloads	**Sensor data, IoT control, dashboards**
URL scheme	http:// / https://	**mqtt:// / mqtts://**
State	Stateless	**Stateful (session, subscriptions)**



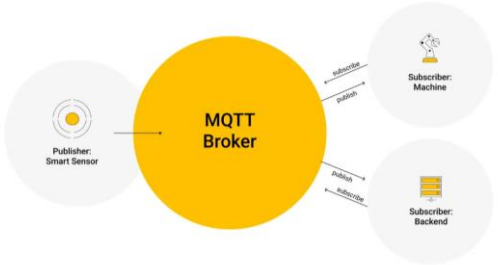
SECTION 02

MQTT Core Concepts

Publish / Subscribe · Topics · Messages · Broker · QoS

Publish / Subscribe Model

The heart of MQTT communication





PUBLISHER

Pico 2W
(BME280 data)

publishes on topic:
pico/temperature

BROKER

HiveMQ / Mosquitto

Filters & routes
messages by topic

Stores retained msgs
Manages sessions

Phone App

Node-RED Dashboard

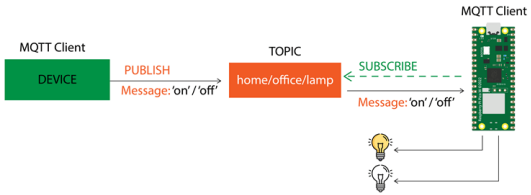
Another Pico 2W

SUBSCRIBERS

All subscribed to:
pico/temperature

MQTT Topics

Hierarchical addressing with forward slashes



Topic Structure

Topics are UTF-8 strings separated by forward slashes (/). Each level narrows the scope:

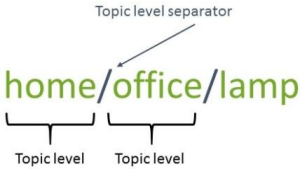
```
home/office/lamp
home/bedroom/temperature
pico/sensor/humidity
pico/led/control
factory/line1/motor/rpm
```

Wildcards

+ (single level)

Matches exactly one level

```
home/+/lamp → matches
home/office/lamp
home/bedroom/lamp
```



(multi level)

Matches any remaining levels

```
home/# → matches
home/office/lamp
home/bedroom/temp
home/kitchen/fan
```

home/office/lamp

≠

home/office/LAmp

⚠ **Topics are CASE-SENSITIVE: Pico/Temp ≠ pico/temp**

MQTT Messages & Quality of Service

QoS levels and message payload

Message Payload

MQTT messages can carry any payload — **plain text**, **JSON**, **binary**, or **sensor values**. The broker delivers them without interpreting content.

```
# Payload examples
client.publish("pico/temp", "24.5")
client.publish("pico/data", '{"t":24.5,"h":60}')
client.publish("pico/led", "ON")
```

Retained Messages

When retain=True, the broker stores the LAST message on that topic. New subscribers instantly receive it without waiting for the next publish.

```
client.publish(topic, msg, retain = True)
```

Quality of Service (QoS)

QoS 0 — At most once

Fire-and-forget. No acknowledgement. May be lost. Lowest overhead.

QoS 1 — At least once

Message acknowledged by receiver. May be delivered more than once.

QoS 2 — Exactly once

4-way handshake. Guaranteed delivery, no duplicates. Highest overhead.

The MQTT Broker

The message routing engine

The broker is the central hub of MQTT. It receives ALL messages, filters by topic, and delivers to all interested subscribers.

HiveMQ Cloud

Cloud-hosted, free plan available. No install needed. Best for beginners and production IoT.

✓ Free tier ✓ TLS built-in ✓ Web UI ✓ No setup
✗ Requires internet access

Mosquitto (Local)

Open-source broker on Raspberry Pi or PC. Most popular in hobbyist and home automation projects.

✓ Free & open-source ✓ Local network
✓ No internet needed ✓ Lightweight
✗ Requires install & configuration

Mosquitto (Cloud VPS)

Mosquitto installed on a cloud server (e.g., DigitalOcean). Accessible worldwide from anywhere.

✓ Internet accessible ✓ Full control
✓ Low cost VPS
✗ Requires Linux admin skills

In this lecture: HiveMQ Cloud (free). The code is compatible with any MQTT broker — just update the connection details.



SECTION 03

Setting Up HiveMQ Broker

Cloud MQTT · Account Setup · Cluster URL · Access Credentials

HiveMQ Cloud Setup

Step-by-step broker configuration

1

Create Account

Go to **hivemq.com** → click '**Start Free**' → choose **HiveMQ Cloud plan** → **register** or **login**.

2

Get Cluster URL

After login, click **'Manage Cluster'**. Copy the **Cluster URL** (e.g. abc123.s1.eu.hivemq.cloud). Save it — you'll need it in your code.

3

Create Credentials

Click '**Access Management**' tab → fill in **Username** and **Password** → **set permission** to *Publish and Subscribe* → click '*Create Credential*'.

4

Collect Details

Before proceeding, make sure you have: ① Cluster URL ② MQTT Username ③ MQTT Password ④ Port: 8883 (TLS)

⚠ **Port 8883 (TLS/SSL) is required for HiveMQ Cloud. Plain port 1883 will not connect. Set ssl=True in your MicroPython MQTT client.**

MQTT Broker Details Checklist

Information needed before writing code

Required Details

MQTT Server / URL:	`abc123.s1.eu.hivemq.cloud`
Port (TLS):	`8883`
Username:	`your_hivemq_username`
Password:	`your_hivemq_password`
Client ID:	`pico2w_01 (unique per device)`

Security Tips

- Store credentials in secrets.py — never hardcode in main.py
- Add secrets.py to .gitignore before pushing to GitHub
- Use TLS (port 8883) — plain text port 1883 exposes credentials
- Use unique Client IDs — duplicate IDs cause disconnection loops
- Rotate passwords periodically in production deployments

Using Mosquitto instead?

Just change the server URL to your Raspberry Pi's local IP (e.g. 192.168.1.10), port to 1883, and set ssl=False. The rest of the code stays identical.



SECTION 04

Installing umqtt Modules

simple.py · robust.py · Thonny IDE file upload · Directory structure

umqtt Library Overview

MicroPython MQTT client modules

umqtt.simple — simple.py

Core MQTT client. Supports connect, publish, subscribe, wait_msg. Lightweight but does NOT auto-reconnect.

Key Methods:

- MQTTClient(id, server)
- connect()
- publish(topic, msg)
- subscribe(topic)
- set_callback(fn)
- wait_msg()

umqtt.robust — robust.py

Extends simple.py with automatic reconnection. Best for long-running IoT deployments where Wi-Fi or broker drops occasionally.

Key Methods:

- Inherits all simple.py methods
- reconnect() — auto-reconnects
- wait_msg() — with reconnect on error
- Suitable for production use

Both modules must be placed in a folder called umqtt/ on the Pico filesystem. Source: github.com/micropython/micropython-lib

Installing umqtt via Thonny IDE

Step-by-step file upload to Pico filesystem

1 Open Files Sidebar

In Thonny IDE → View → Files. You'll see two panels: your computer (top) and Pico 2W filesystem (bottom).

2 Create umqtt Directory

Right-click on the Pico 2W panel → New Directory → type umqtt → click OK. The folder appears in the sidebar.

3 Create simple.py

Right-click on the umqtt folder → New File → name it simple.py. Paste the code from the GitHub link (see below).

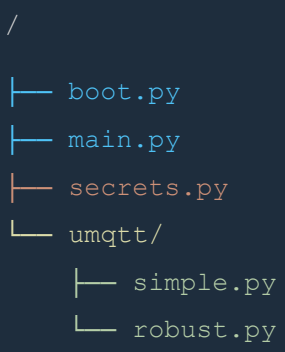
4 Create robust.py

Repeat step 3 for robust.py. Paste the robust.py code. Both files must be inside the umqtt/ folder.

5 Verify Structure

In Thonny's Pico panel you should see: /umqtt/simple.py and /umqtt/robust.py. You're ready to import!

Pico 2W Filesystem



⚠ **Download links: github.com/RuiSantosdotme/Random-Nerd-Tutorials → Projects/Raspberry-Pi-Pico/MicroPython/mqtt/umqtt/**



SECTION 05

Configuration & Boot Files

secrets.py · boot.py · Wi-Fi Connection · MQTT Credentials

Creating secrets.py

Keeping credentials out of your main code

```
# secrets.py — NEVER share this file publicly!
# Add secrets.py to .gitignore before using Git

# Wi-Fi credentials
WIFI_SSID = "Your_WiFi_Name"
WIFI_PASSWORD = "Your_WiFi_Password"

# MQTT broker details (HiveMQ example)
MQTT_SERVER = "abc123.s1.eu.hivemq.cloud"
MQTT_PORT = 8883
MQTT_USER = "your_hivemq_username"
MQTT_PASSWORD = "your_hivemq_password"
MQTT_CLIENT_ID = "pico2w_01"
```

Importing in your scripts

```
# In boot.py / main.py:
from secrets import (
    WIFI_SSID, WIFI_PASSWORD,
    MQTT_SERVER, MQTT_PORT,
    MQTT_USER, MQTT_PASSWORD,
    MQTT_CLIENT_ID
)
```

Then use WIFI_SSID, MQTT_SERVER, etc. as normal variables throughout your code.

Security Best Practices

- Never hardcode credentials directly in main.py or publish.py
- Create a .gitignore file with: secrets.py
- Import from secrets: from secrets import WIFI_SSID, MQTT_SERVER, ...

Mosquitto users:

Set MQTT_PORT = 1883 and ssl = False in the connect call. If no password is required, pass user = None and password = None.

boot.py — Wi-Fi Connection

Auto-run on power-up; connects Pico 2W to Wi-Fi

```
# boot.py — runs automatically on every boot
import network, time
from secrets import WIFI_SSID, WIFI_PASSWORD

def connect_wifi():
    wlan = network.WLAN(network.STA_IF)
    wlan.active(True)
    if not wlan.isconnected():
        print("Connecting to Wi-Fi...")
        wlan.connect(WIFI_SSID, WIFI_PASSWORD)
        timeout = 0
        while not wlan.isconnected() and timeout < 20:
            time.sleep(1)  ;  timeout += 1
            print(".", end="")
    if wlan.isconnected():
        print("\nConnected!")
        print("IP:", wlan.ifconfig()[0])
    else:
        print("Wi-Fi FAILED")
return wlan

connect_wifi()  # Called on boot automatically
```

boot.py Behavior

Auto-runs first

boot.py runs before main.py on every power-up or reset.

Network ready

By the time main.py runs, Wi-Fi is already connected.

Reusable

Import connect_wifi() from boot if needed in main.

Timeout guard

20-second loop prevents infinite hang on bad credentials.

Status codes

STAT_GOT_IP = 3 = success. -2 = no AP. -3 = wrong password.

MQTT Client Connection Helper

Connecting to the broker in MicroPython

```
\ \ \
from umqtt.simple import MQTTClient
from secrets import MQTT_SERVER, MQTT_PORT, MQTT_USER, MQTT_PASSWORD, MQTT_CLIENT_ID

def connect_mqtt():
    client = MQTTClient(
        client_id  = MQTT_CLIENT_ID,
        server     = MQTT_SERVER,
        port       = MQTT_PORT,
        user       = MQTT_USER,
        password   = MQTT_PASSWORD,
        keepalive  = 60,
        ssl        = True,         # Required for HiveMQ (TLS port 8883)
        ssl_params = {"server_hostname": MQTT_SERVER}
    )
    client.connect()
    print("Connected to MQTT broker")
    return client
```

MQTTClient Parameters

param	description
`client_id`	Unique ID for this device
`keepalive`	Ping interval in seconds (0=disabled)
`ssl=True`	Enable TLS — required for port 8883

Mosquitto (local)?

Set port=1883, ssl=False, remove ssl_params. If anonymous access, set user=None and password=None. Everything else stays the same.



SECTION 06

Publishing MQTT Messages

Sensor Data · BME280 · Topics · JSON Payloads · Retained

Reading BME280 Sensor

Temperature · Humidity · Pressure over I²C

```
\ \ \
# Install BME280 library first:
# Thonny → Tools → Manage Packages → 'bme280'
from machine import Pin, I2C
import bme280, time

# I2C setup: SDA=GP4, SCL=GP5
i2c = I2C(0, sda=Pin(4), scl=Pin(5), freq=400000)
bme = bme280.BME280(i2c=i2c)

# Read sensor values
values = bme.values
# values returns: ('24.50C', '60.23%', '1013.25hPa')

# Or read raw floats:
temp = round(bme.temperature, 2)
hum = round(bme.humidity, 2)
pres = round(bme.pressure, 2)
print(f'Temp: {temp}°C  Hum: {hum}%  Pres: {pres} hPa')
```

BME280 Quick Facts

Parameter	Value
Interface	I²C or SPI
Address	0x76 or 0x77
Temp range	-40 to +85 °C
Hum accuracy	±3% RH
Pressure	300–1100 hPa
Pico Pins	GP4 (SDA), GP5 (SCL)

Install BME280 library: Thonny → Tools → Manage Packages → search 'bme280' → install 'micropython-bme280'.

Complete Publish Script — publish.py

Sending BME280 sensor data to MQTT broker

```
\\\
# publish.py — publish BME280 sensor data every 10 seconds
import time, json
from machine import Pin, I2C
import bme280
from umqtt.simple import MQTTClient
from secrets import MQTT_SERVER, MQTT_PORT, MQTT_USER, MQTT_PASSWORD, MQTT_CLIENT_ID

# Topics
TOPIC_TEMP = "pico/bme280/temperature"
TOPIC_HUM = "pico/bme280/humidity"
TOPIC_PRESS = "pico/bme280/pressure"

# Hardware
i2c = I2C(0, sda=Pin(4), scl=Pin(5), freq=400000)
bme = bme280.BME280(i2c=i2c)

# Connect to broker
client = MQTTClient(MQTT_CLIENT_ID, MQTT_SERVER, MQTT_PORT,
                    MQTT_USER, MQTT_PASSWORD, ssl=True, ssl_params={"server_hostname": MQTT_SERVER})
client.connect()
print("MQTT connected!")

while True:
    temp  = str(round(bme.temperature, 2))
```

Publish Code — Deep Dive

Understanding each part of publish.py

Topic Design

```
pico/bme280/temperature
```

Follow a hierarchy: device/sensor/measurement. Consistent naming makes dashboards and filtering much easier to manage.

MQTTClient()

```
client = MQTTClient(id, server, port, user, pwd, ...)
```

Creates the client object. Does NOT connect yet — just configures parameters. Call .connect() separately.

client.connect()

```
Initiates TCP + MQTT handshake with the broker
```

Sends CONNECT packet. Broker replies with CONNACK. If credentials are wrong, raises MQTTException.

client.publish()

```
publish(topic, msg, retain=True)
```

Sends PUBLISH packet. retain=True stores last value on broker — new subscribers get it immediately without waiting for next publish.

time.sleep(10)

```
Controls publish frequency
```

10 seconds is a reasonable interval for environment sensors. Adjust based on how fast values change and broker rate limits.

Publishing JSON Payloads

Structured data for dashboards and parsers

```
\ \ \
# Option A: Separate topics per measurement
client.publish("pico/temp", "24.5")
client.publish("pico/hum", "60.1")

# Option B: Single JSON topic (recommended)
import json
payload = json.dumps({
    "temperature": temp,
    "humidity": hum,
    "pressure": pres,
    "device": "pico2w_01"
})
client.publish("pico/sensors", payload)

# Example published JSON:
# {"temperature": 24.5, "humidity": 60.1,
#  "pressure": 1013.2, "device": "pico2w_01"}
```

JSON vs Flat Topics

	Flat Topics	JSON Payload
Topics	3 publishes	**1 publish**
Parsing	Simple	`json.loads()`
Node-RED	Multiple nodes	**JSON node**
Best for	Simple sensors	**Multi-value**

Home Assistant & Adafruit IO:

These platforms expect specific MQTT topic formats. Check their documentation for the exact topic structure and payload format.

Tip: Use json.loads(msg) on the subscriber side to parse incoming JSON. msg will be a bytes object — decode first: msg.decode()



SECTION 07

Subscribing to MQTT Topics

Callbacks · set_callback() · wait_msg() · LED Control

Subscribe — How It Works

Receiving messages from the broker

1

`set_callback(fn)`

Register a Python function that fires when a message arrives on any subscribed topic.

2

`client.connect()`

Connect to broker and establish persistent TCP session.

3

`client.subscribe(topic)`

Send SUBSCRIBE packet to broker. Broker notes this client wants topic messages.

4

`wait_msg() / check_msg()`

wait_msg() BLOCKS until a message arrives. check_msg() is non-blocking (returns None if no message).

5

`Callback fires`

When message arrives, broker sends PUBLISH → client calls your callback function with (topic, msg).

6

`Act on message`

Inside callback: decode msg, check content, control GPIO, update display, etc.

Complete Subscribe Script — subscribe.py

Controlling Pico LED from MQTT messages

```
\\\
# subscribe.py — receive commands to control onboard LED
import time
from machine import Pin
from umqtt.simple import MQTTClient
from secrets import MQTT_SERVER, MQTT_PORT, MQTT_USER, MQTT_PASSWORD, MQTT_CLIENT_ID

# Hardware
led = Pin("LED", Pin.OUT)  # onboard LED

# Topic to subscribe to
TOPIC_LED = "pico/led/control"

# Callback function — called when a message arrives
def callback(topic, msg):
    print(f'Received: {topic} → {msg}')
    if msg == b"ON" or msg == b"1":
        led.on()
        print("LED turned ON")
    elif msg == b"OFF" or msg == b"0":
        led.off()
        print("LED turned OFF")

# Connect and subscribe
client = MQTTClient(MQTT_CLIENT_ID, MQTT_SERVER, MQTT_PORT,
```

Subscribe Code — Deep Dive

Understanding every key line

`set_callback(callback)`

Must be called BEFORE connect(). The callback receives (topic: bytes, msg: bytes). Both are byte strings — compare with b"ON", not "ON".

`msg == b"ON"`

Messages arrive as bytes objects. Use b"..." literals or msg.decode() then compare strings. Never compare with plain strings directly.

`client.subscribe(topic)`

Registers interest with broker. Multiple calls allowed for different topics. One callback handles all — check topic inside callback.

`wait_msg()`

Blocks until ONE message arrives and calls callback. Use in a while True loop. For non-blocking: use check_msg() — returns None if no message.

`check_msg()`

Non-blocking alternative. Use when you have other tasks to run between checks. Call frequently (e.g., every 100ms) to avoid missing messages.

To subscribe to MULTIPLE topics: call client.subscribe() multiple times. Inside callback, check topic parameter to route messages correctly.

Combined Publish + Subscribe Script

One Pico that both reads sensors AND receives commands

```
\ \ \
# main.py — publish sensor data AND subscribe to LED control
import time, json
from machine import Pin, I2C
import bme280
from umqtt.robust import MQTTClient
from secrets import *

led = Pin("LED", Pin.OUT)
i2c = I2C(0, sda=Pin(4), scl=Pin(5), freq=400000)
bme = bme280.BME280(i2c=i2c)

def on_message(topic, msg):
    if topic == b"pico/led/control":
        led.value(1 if msg == b"ON" else 0)

client = MQTTClient(MQTT_CLIENT_ID, MQTT_SERVER, MQTT_PORT,
    MQTT_USER, MQTT_PASSWORD, ssl=True, ssl_params={"server_hostname": MQTT_SERVER})
client.set_callback(on_message)
```

Key Design Pattern

Use check_msg() (non-blocking) instead of wait_msg() when publishing too. check_msg() returns immediately so you can run other tasks. Track publish timing with ticks_ms() difference — no blocking sleep() needed.

```
    client.check_msg()        # Non-blocking — check for incoming messages
    if time.ticks_diff(time.ticks_ms(), last_publish) > 10000:
```

umqtt.robust — Auto-Reconnect

Production-grade MQTT with reconnection handling

Problem with umqtt.simple

If the Wi-Fi or broker connection drops, simple.py raises an OSError exception and the script crashes. In a long-running IoT deployment, this is unacceptable.

```
# simple.py behavior on drop:
# OSError: [Errno 104] ECONNRESET
# Script halts — needs manual reset
# Bad for production deployments!
```

Symptoms:

- Pico stops publishing after broker restart
- No data on dashboard until manual Pico reset
- MQTT client disappears from broker list

Solution: umqtt.robust

robust.py extends simple.py with automatic reconnection. If the connection drops, it re-connects to the broker and re-subscribes automatically.

```
# Just change the import!
# FROM:
from umqtt.simple import MQTTClient
# TO:
from umqtt.robust import MQTTClient
```

Benefits:

- Reconnects automatically on Wi-Fi drop
- Re-subscribes to all topics after reconnect
- No script changes beyond the import line

→ Always use umqtt.robust for production and long-running deployments. Use umqtt.simple only for learning and quick experiments.



SECTION 08

Testing & Troubleshooting

MQTT Explorer · HiveMQ WebUI · Debug Tips · Common Errors

Testing with MQTT Explorer

Visual tool to monitor topics and publish test messages

1 **Download**

Get MQTT Explorer free from: mqtt-explorer.com. Available for Windows, macOS, Linux.

2 **Connect**

Enter broker URL, port 8883, username, password, enable TLS. Click Connect.

3 **Monitor Topics**

All topics published by your Pico 2W appear in the left panel as a tree. Click any topic to see its current value and history.

4 **Publish Test**

Right side: enter topic (e.g. pico/led/control), type payload (ON or OFF), click Publish. Your Pico receives it instantly.

5 **Verify**

Check Thonny Shell on Pico side — the callback print() should show the received message confirming end-to-end success.

Other Test Tools

HiveMQ Web Client

Built-in browser client at hivemq.com/demos/websocket-client/

MQTT.fx

Desktop client, good for scripted testing

mosquitto_pub / _sub

CLI tools — fast for command-line testing

Node-RED

Flow-based dashboard with MQTT nodes

Home Assistant

Full HA integration with MQTT Discovery

Troubleshooting Common Issues

Symptoms, causes, and fixes

Symptom	Likely Cause	Solution
OSError: ECONNRESET	MQTT connection dropped (Wi-Fi or broker restarted)	**Switch to umqtt.robust for auto-reconnect**
MQTTException: 5	Wrong username or password for broker	**Double-check credentials in secrets.py**
OSError: -2	Hostname not resolved (DNS failure or wrong URL)	**Verify cluster URL, check Wi-Fi is connected first**
Script hangs on connect()	Port mismatch: using 1883 on TLS-only broker	**Use port 8883 with ssl=True for HiveMQ**
Callback never fires	set_callback() called after connect()	**Always call set_callback() BEFORE connect()**
msg comparison fails	Comparing msg == 'ON' instead of b'ON'	**Messages are bytes — use b'ON' or msg.decode()**
Only publishes once	wait_msg() inside publish loop blocks indefinitely	**Use check_msg() for non-blocking message checks**
Broker disconnects Pico	Duplicate Client ID on same broker	**Use unique MQTT_CLIENT_ID per physical device**

Debugging Tips & Best Practices

Making MQTT development easier

Print broker responses

Add try/except around connect() and print the exception. MQTTException code 5 = bad credentials; 128+ = protocol error.

Verify Wi-Fi before MQTT

Print wlan.isconnected() and wlan.ifconfig() before attempting MQTT connect. No Wi-Fi = connection will always fail.

Test topics in isolation

Use MQTT Explorer to publish to pico/led/control manually BEFORE running subscribe script. Isolates server vs client issues.

Use retained messages

Publish with retain=True so MQTT Explorer shows the LAST value immediately when you connect — great for debugging state.

Check Thonny Shell

All print() output from Pico appears in Thonny Shell. Add verbose prints in callback to trace every received message.

Binary vs string compare

Print type(msg) inside callback. It will show <class 'bytes'>. Use msg.decode() == 'ON' or msg == b'ON'.

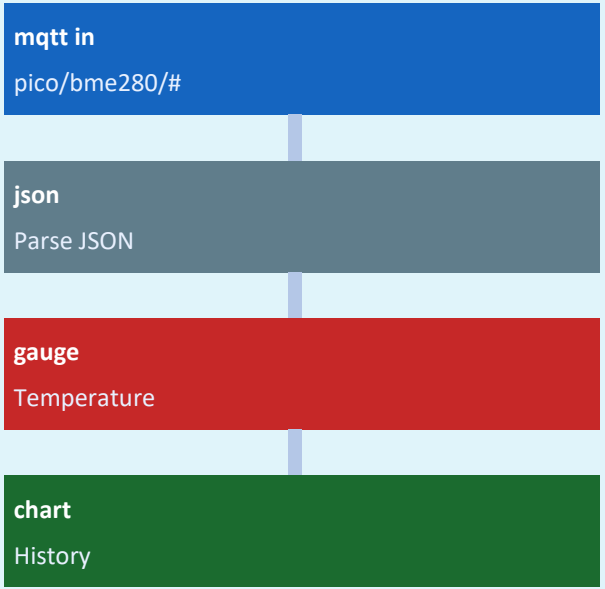
MQTT with Node-RED Dashboard

Visualizing Pico sensor data in real time

Node-RED MQTT Setup

1	**Install Node-RED**	npm install -g node-red (on Raspberry Pi or PC)
2	**Open browser**	Navigate to http://localhost:1880
3	**Add MQTT-in node**	Drag 'mqtt in' node. Configure broker URL, port, credentials.
4	**Set topic**	Set topic to pico/bme280/# (wildcard for all sensor readings)
5	**Add dashboard**	Install node-red-dashboard, add 'gauge' or 'chart' nodes
6	**Connect & Deploy**	Wire MQTT-in → JSON → Gauge. Click Deploy. Open /ui dashboard.

Node-RED Flow



Other MQTT Platforms:

Home Assistant · Adafruit IO · OpenHAB · Grafana + InfluxDB

Home Assistant MQTT Integration

Connecting Pico 2W to home automation

```
# MQTT Discovery for Home Assistant
# HA auto-detects device if you publish to correct topic
import json

# Step 1: Publish device discovery config
config_topic = "homeassistant/sensor/pico_temp/config"
config_payload = json.dumps({
    "name": "Pico Temperature",
    "device_class": "temperature",
    "state_topic": "pico/bme280/temperature",
    "unit_of_measurement": "°C",
    "unique_id": "pico2w_temp_01"
})
client.publish(config_topic, config_payload, retain=True)

# Step 2: Publish sensor data as usual
# home/pico/bme280/temperature → HA reads it automatically
client.publish("pico/bme280/temperature", str(temp),
retain=True)
```

HA MQTT Discovery

- Enable MQTT integration in HA Settings → Devices
- Configure broker URL (same HiveMQ or local Mosquitto)
- Publish config payload to homeassistant/<domain>/<id>/config
- HA auto-creates entity — appears in dashboard immediately
- State updates flow via state_topic — same as regular publish

MQTT Discovery eliminates manual HA entity config. Publish the config JSON once and HA auto-discovers your Pico as a device with all its sensors.

Advanced: Last Will & Testament (LWT)

Knowing when your Pico goes offline

What is LWT?

LWT lets you specify a message that the broker publishes AUTOMATICALLY if your Pico disconnects ungracefully (power loss, Wi-Fi drop, crash). Other subscribers receive the offline notice immediately.

```
# Set LWT before calling connect()
client.set_last_will(
    topic    = "pico/status",
    msg      = "offline",
    retain   = True,
    qos      = 1
)
client.connect()  # LWT registered in CONNECT packet
```

When Pico connects cleanly, it publishes 'online' itself. The broker sends 'offline' only on ungraceful disconnect.

LWT Use Cases

Device presence

Dashboard shows online / offline per device

Alert system

Node-RED sends email if device goes offline unexpectedly

HA availability

Home Assistant uses LWT to mark entity as unavailable

Fleet monitoring

Track which of many Pico devices are currently active

```
# On clean boot: announce online status
client.publish("pico/status", "online", retain=True)
```

Summary & Key Takeaways

What you've learned in this lecture

MQTT Protocol

Lightweight pub/sub over TCP. RFC 6455. Ports 1883 (plain) / 8883 (TLS). Minimal overhead — perfect for IoT.

secrets.py

Store all credentials in secrets.py. Never hardcode in main script. Add to .gitignore.

Core Concepts

Publish, Subscribe, Topics (hierarchical, case-sensitive), Messages (any payload), Broker (filters & routes).

Publishing

MQTTClient → connect() → publish(topic, msg, retain=True). Use JSON for multi-value payloads.

Broker Setup

HiveMQ free cloud (no install), Mosquitto local (Raspberry Pi), or cloud VPS. Need URL, port, username, password.

Subscribing

set_callback() BEFORE connect(). subscribe(topic). wait_msg() blocks; check_msg() non-blocking.

umqtt Modules

simple.py = basic MQTT client. robust.py = auto-reconnect. Install in /umqtt/ folder on Pico filesystem.

Production Tips

Use robust.py, unique client IDs, LWT for offline detection, check_msg() for multi-task loops.

MicroPython MQTT — Quick Reference Cheat Sheet

Raspberry Pi Pico 2W · umqtt.simple / umqtt.robust

```
# Connect to broker
from umqtt.robust import MQTTClient
client = MQTTClient(client_id, server,
    port, user, pwd, ssl=True,
    ssl_params={"server_hostname": server})
client.set_callback(callback_fn)
client.connect()
```

```
# Callback template
def callback(topic, msg):
    t = topic.decode()
    m = msg.decode()
    if t == "pico/led":
        led.value(1 if m=="ON" else 0)
```

```
# Publish
client.publish(topic, msg)
client.publish(topic, msg, retain=True)

# Subscribe
client.subscribe("home/+/lamp")
# Wait: blocking / non-blocking
client.wait_msg()
client.check_msg()
```

```
# Pub + Sub main loop
last = 0
while True:
    client.check_msg()  # non-blocking
    if time.ticks_diff(time.ticks_ms(), last) >
10000:
        client.publish(topic, data, retain=True)
        last = time.ticks_ms()
```

```
# LWT: client.set_last_will(topic, 'offline', retain=True, qos=1)  — BEFORE connect()
```

Review Questions & Lab Checklist

Verify your understanding

Review Questions

- What does MQTT stand for, and what is its primary design goal?
- Explain the publish/subscribe model. How does it differ from HTTP?
- What is the role of the MQTT broker? Name two broker options.
- What are MQTT topic wildcards? Give an example of + and # usage.
- What is the difference between QoS 0, 1, and 2?
- Why must set_callback() be called BEFORE connect()?
- What is the difference between wait_msg() and check_msg()?
- Why use umqtt.robust instead of umqtt.simple in production?
- What happens if two devices share the same MQTT Client ID?
- How do MQTT retained messages benefit late-joining subscribers?

Lab Checklist

- [ ] HiveMQ account created & cluster URL saved
- [ ] umqtt/simple.py uploaded to Pico
- [ ] umqtt/robust.py uploaded to Pico
- [ ] secrets.py created with correct credentials
- [ ] boot.py connects Pico 2W to Wi-Fi
- [ ] publish.py sends BME280 data successfully
- [ ] subscribe.py controls Pico LED from MQTT
- [ ] MQTT Explorer shows published topics
- [ ] Multi-client test: two subscribers update together
- [ ] Bonus: Node-RED or HA dashboard running







End of Lecture

Raspberry Pi Pico 2W — Getting Started with MQTT

MicroPython · umqtt · HiveMQ · Pub/Sub · BME280 · Node-RED

You can now connect any Pico 2W sensor to any MQTT-enabled platform!